

Real Estate AI Agent Module Documentation

Real Estate AI Analyzer Module Documentation

Version: 1.1

Architecture: Statistical Analysis & LLM Context Generation

Primary Objective: To provide rigorous, statistically sound insights (Fair Price, Market Pulse, Investment Gems) based on highly clean data, preventing LLM hallucination through hardcoded math.

1. System Architecture Overview

The `3.Analyzer` module is the mathematical backbone of the AI system. It does not guess or hallucinate. Instead, it computes hard statistics on local property markets and converts those numbers into rigid text templates that the LangGraph Agent (`1.Agent`) can safely read aloud to the user.

1.1 Layered Design

- **Layer 1: General Market Engine** (`3_market_engine.py`) - Looks at the whole city or district (macroscopic).
- **Layer 2: Preference Engine** (`4_preference_engine.py`) - Looks only at the specific properties the Recommender just suggested to the user (microscopic).
- **Layer 3: Insight Builder** (`5_insight_builder.py`) - Packages the raw numbers into LLM-safe strings.
- **Backtesting Layer** (`6_analyzer_evaluator.py`) - A rigorous QA script that validates the mathematical precision of the estimates.

2. Ingestion & Configuration

2.1 The Data Contract (`2_ingestion_adapter.py`)

To prevent the Recommender and the Analyzer from showing conflicting numbers (Logical Drift), the Analyzer **does not clean its own data**.

Instead, it imports the `preprocessing_engine.py` from the `4.Data` directory to process data exactly as the Recommender does. It caches the resulting pristine dataset in RAM (`_GLOBAL_ANALYZER_DATAFRAME_CACHE`) for near-instant access across all subsequent mathematical queries.

2.2 Global Thresholds (`1\_config.py`)

- `min_sample_size` (10): The module refuses to generate a "Fair Price" if fewer than 10 comparable properties exist, protecting the user from wildly inaccurate predictions based on 1 or 2 outlier properties.
- `premium_iqr_multiplier` (1.5): Standard statistical threshold for stripping extreme luxury outliers from median calculations.

3. General Market Engine (`3\_market\_engine.py`)

This engine calculates broad, objective truths about the real estate landscape.

3.1 Fair Price Estimator

The most critical mathematical component in the system. Given an asking price, it evaluates whether it is a good deal.

The Algorithm:

1. **Hierarchical Masking:** Drills down to exact comparables (Same Category -> Same Town -> Same District -> Same Property Type).
2. **Size Bucketing:** Filters for properties with the same number of bedrooms and +/- 20% total area.
3. **Cascading Fallbacks:** If the sample size is too small (<5), it gracefully expands the search radius (e.g., drops the District requirement, or allows +/- 1 bedroom).
4. **Mathematical Trimming:** Calculates `price_per_sqm`, strips the extreme top/bottom 5% tails, and then applies an Interquartile Range (IQR) filter to remove skewed outliers.
5. **Output Generation:**
 - Calculates a `predicted_fair_price` based on the robust median.
 - Calculates a `confidence_score` (penalizing low sample sizes or violently high variance).
 - Assigns a categorical verdict: `highly_competitive`, `fair_value`, `premium_priced`, or `overpriced`.

3.2 Investment Scorer (Gems)

Scans an entire town for properties priced >15% below the market median for their specific bedroom count. It isolates these "undervalued" properties and returns them as prime investment opportunities.

3.3 Market Pulse & Area Comparator

- **Market Pulse:** Generates macroscopic statistics like Total Active Listings, Buy/Rent distribution, Median Prices, and the "Liquidity Velocity" (`median_days_on_market`) to tell users how fast properties are selling.
- **Area Comparator:** Compares two competing towns side-by-side on metrics like Price Differences, Off-Plan Percentages, and Listing Counts.

4. Preference Engine (`4\_preference\_engine.py`)

While the Market Engine looks at **everything**, the Preference Engine looks only at **you**.

4.1 "Your Market" Segment Report

It loads the `Rec_To_Ana_Candidates.json` payload dumped by the Recommender. It then calculates the exact median price, area, and payment mix specifically for the Top 10 properties the AI just presented to the user.

4.2 Trade-off Advisor

Uses "What-If" logic on the global database to counsel the user.

- **Scenario A:** It simulates a 20% budget increase. If this unlocks significantly more properties, it advises the user to increase their budget.
- **Scenario B:** It drops the bedroom constraint entirely and checks if that yields a wider pool.

5. Insight Builder (`5\_insight\_builder.py`)

LLMs are notoriously bad at math. To solve this, the `Insight Builder` strips the LLM of its computational responsibilities.

It takes the JSON output from the Market Engine and formats it into strict, unalterable text blocks.

Example Context Block:

[FAIR_PRICE_CONTEXT]

- Query: 3 bed Apartment in New Cairo City
- Asking Price: 5,200,000 EGP
- Market Median: 4,950,000 EGP
- Comparable Data Points: 182
- Price Percentile: 65th percentile
- Analyst Verdict: PREMIUM PRICED
- Statistical Confidence: 92%

The `LangGraph Tongue` node simply reads this block and weaves it into a conversational sentence.

6. Analyzer Evaluator (`6\_analyzer\_evaluator.py`)

A rigorous Quality Assurance script that proves the math works.

It runs a **Leave-One-Out (LOO)** Backtest:

6. Picks 100 properties.
7. Hides property #1's actual price from the Estimator.

8. Asks the Estimator to predict the Fair Price of property #1 using the rest of the database.
9. Compares the Prediction to the Hidden Actual Price.
10. Calculates the **Median Absolute Percentage Error (MdAPE)**. If the error is $>15\%$, the build fails.